# Project 1 — The Grader

CS496 AI Engineering · Mediterranean Institute of Technology · Fall 2026
Team: Nadia Ben Njima, Ahmed Brini, Rahma Sbei, Mariem Lahmer · Repository state: 27 September 2026

## 1. The task, and why exact match is not enough

**Task:** given a short *brief* (recipient, relationship, purpose, 3–6 key points, optional sender), write one professional email that follows a 12-rule style guide. The rules are in four bands: **structure** (greeting, body, sign-off), **tone** (professional register, no slang, formality matching the relationship), **content** (no invented facts, all key points addressed, correct names) and **form** (grammatical, concise, well paragraphed). Each email also gets an overall score from 1 to 5.

**Why exact match fails:** for any brief there are many correct emails with different wording, order and length. A string or keyword comparison cannot tell "the delay was caused by a vendor issue" from "one of our suppliers let us down". It also cannot judge whether the tone fits a client or a teammate. The task passes all five of the brief's tests: no single correct answer, rules that fit on two pages, a labelled set buildable in a week, one model call per item, and reuse in later projects. So quality has to be graded by a model, and that model has to be shown to agree with humans.

## 2. The golden set

**Construction:** 15 realistic scenarios, each rendered as 3–4 candidate emails: one clean email, plus 2–3 with one deliberately injected violation. That gives **50 items**. Injected types: none 15, tone 10, content 10, structure 9, form 6. Split: **35 dev** (used to iterate prompts) and **15 test** (used once, for the final numbers). Stored as `data/golden_set.jsonl`. The rules and examples are in `data/labelling_guide.md`.

**Labelling:** two people labelled every item independently (12 booleans + 1–5 score). We measured agreement on the raw labels, then reconciled disagreements into one `reconciled_label` with a written `resolution_note`. The reconciled label is the ground truth for the judge. **0 items were dropped.** Size note: the brief asks for 150+ items; we kept a 50-item set as a team decision, and the postmortem records it as a gap.

| Inter-labeller agreement (n=50, before reconciliation) | Result |
|---|---|
| All 600 rule labels | 590/600 (98.3%) |
| Items with any disagreement | 10/50 (20%), all dev |
| Overall score, exact / within 1 | 40/50 (80%) / 50/50 |
| Overall score, Cohen's κ | 0.691 |

| Rules with any disagreement | Agree | κ |
|---|---|---|
| form.well_paragraphed | 43/50 (86%) | 0.19 |
| content.correct_names | 48/50 (96%) | 0.00* |
| form.concise | 49/50 (98%) | 0.91 |
| other 9 rules | 50/50 (100%) | 1.00 / n/a* |

* κ is undefined (n/a) or close to 0 when nearly every label has the same value, so percent agreement is the headline number. Computed by `python -m harness.runner label-agreement`. In all 10 disagreements the reconciled score took the **lower** of the two scores, so the human standard leans strict.

## 3. The harness

- **One command per mode** (`harness/runner.py`): `label-agreement`, `validate-judge` (judge vs reconciled labels), `check-judge-bias`, `run-system` (system writes, judge grades). The `--split dev|test` and `--limit` options apply to every mode; `--start` resumes a stopped bias check.
- **One row per item** in `results/*.jsonl`: input, output, judge verdict (12 booleans, score, reason), deterministic `rule_checks`, and `_meta` (model id, latency, tokens). Each run also writes a summary JSON with counts and rates: per-rule pass rate or agreement, score distribution, a per-brief-tag breakdown, latency and tokens.
- **Structured output:** both models run in JSON mode, and both outputs are validated against fixed schemas (`schema.py`). A malformed or non-object response becomes a recorded error row, never a crash or a silent pass. Every item has its own error guard, and partial results are written on Ctrl-C.
- **Two clients, two model families:** the judge (`llm_client.py`) runs `openai/gpt-oss-120b` and the system (`llm_client_sys.py`) runs `qwen/qwen3.8-27b`, as explained in §5.
- **System briefs** (`data/briefs.jsonl`): 30 distinct briefs, none shared with the golden set, split 20 dev / 10 test. Each brief is tagged with what it tests: exact numbers (14), many key points (12), "do not…" constraints (7), sender given (6), unknown recipient name (3), plus tone cases. The tags are never shown to either model.
- **Prompts under git:** `prompts/judge_prompt_v1..v3.txt`, `system_prompt_v1..v2.txt`. `prompts/CHANGELOG.md` links every change to its before/after score.

## 4. The judge — reliability report

### 4.1 Prompt versions and agreement with humans

| Version | Change (targeted at inspected mismatches) | n / split | Score exact | Within 1 | Weakest rules |
|---|---|---|---|---|---|
| v1 | Rules copied from the labelling guide; 12 booleans + score + reason | 50 / all | 42% | 76% | formality 0.72, grammatical 0.72 |

| | | | | | |
|---|---|---|---|---|---|
| v2 | correct_names fails only on a *wrong* name; grammar excludes casual register; explicit relationship→formality mapping; "score rules independently"; padding examples | 35 / dev | 31.4% | 74.3% | greeting, sign_off, formality, key points 0.74 |
| **v3** | Score bands tied to specific rule failures; greeting/sign-off "false only if missing"; harsh tone moved to formality; implied key points count; concrete paragraph test | 35 / dev | **34.3%** | **94.3%** | addresses_all_points 0.77 |

v3 per-rule agreement on dev: 11 of 12 rules at 83–100%, with `addresses_all_points` at 77%. Its root cause is confirmed on 6/6 mismatches: when an email states a key point *wrongly*, the judge marks the point missing, while humans count it as addressed and fail only no_invented_facts. The biggest single gain (within-1 from 74% to 94%) came from tying the score bands to rule failures. Exact score agreement stays low partly because the humans themselves agree exactly on only 80% of scores. **Decision: the judge is locked at v3 on gpt-oss-120b.** Any change to the prompt or the model reopens the lock. The final `validate-judge --split test` number on the 15 held-out items is **still to be run**.

### 4.2 Bias checks

**Verbosity bias:** each email is graded as written, then again with four content-free filler sentences inserted before the sign-off. **Position bias:** our judge grades one email at a time, so there are no two answers to swap. The closest equivalent is to swap the order of the BRIEF and CANDIDATE EMAIL blocks in the prompt and check whether the verdict changes.

| Probe (judge v3, golden items 1–7) | Score changed |
|---|---|
| Padding raised the score (verbosity bias) | 0/7 (0%) |
| Swapping BRIEF/EMAIL order changed the score (order bias) | 0/7 (0%) |

> **Limits of this result.** The run was planned for 20 items but stopped at 7 because of API rate limits. The first version of the check made 4 calls per item, and a validation job was running against the same model at the same time. The per-item scores were not saved; the file was rebuilt from the console output, and the unknown fields are `null`. We do not know whether padding *lowered* the score, which is the desired behaviour, or whether single rules flipped. There is no no-change noise baseline, and 2 of the 7 items are test items. The check now uses 3 calls per item, saves partial results and can resume (`--start 8 --limit 13`).

### 4.3 Where the judge fails (demo-day warnings)

- **Lenient on subtle additions.** It passed 8/8 emails with unfilled placeholders (`Dear [Client Name]`, `[Your Name]`) and invented commitments ("We have since resolved the issue", "we will notify you immediately"). The golden set only contains loud, injected violations, so its agreement numbers do not cover this. The deterministic placeholder check (`rule_checks.py`) now reports these misses separately.
- **False "wrong name" failures.** It failed correct formal greetings ("Dear Ms. Ben Amor", "Dear Mr. Dridi") for "omitting the first name", and called "Hi Sarra" misspelled when it matched the brief exactly. These were 1/20 dev and 2/10 test items, and they are every sub-5 score in the final system runs.
- **Rubric gap and noise.** v3's score bands give no band for a correct_names failure, so the same non-defect was scored 2 in one email and 4 in another. The identical greeting also went from 4 to 5 between two runs.
- **Prompt injection.** The candidate email is placed inside the judge's system message, and the only defence is an instruction.

## 5. The system — same-model bias, prompt versions, results

### 5.1 Same-model bias and how we fixed it

In the first version, `system.py` and `judge.py` shared one client, so **the judge was grading emails written by its own model** (gpt-oss-120b). The first run scored **5.0 on 50/50 items and 12/12 rules**. That is not credible, because the same judge scored human-written emails anywhere from 1 to 5. First fix: a separate system client pointed at `gpt-oss-20b`. It still scored 5.0 everywhere, because 20b and 120b are the **same family** (OpenAI gpt-oss: same training recipe and house style), and self-preference works at the family level. Final fix: the system moved to **qwen/qwen3.8-27b (Alibaba)**. Llama models are Enterprise-only on Groq, and Qwen is a preview model, which is a risk we note. We moved the system rather than the judge because the judge had already been validated and locked. An audit found that the model was not the only cause. The system briefs were the golden set's 15 scenarios repeated 3–4 times, so there were only about 15 distinct emails. The briefs were easy (3 points each) and overlapped with the judge's tuning data. We replaced them with the 30 distinct, harder briefs described in §3.

### 5.2 Score history

| Run | System prompt / model | Briefs | n | Judge avg | Distribution | Status |
|---|---|---|---|---|---|---|
| 1 | v1 / gpt-oss-120b (= judge model) | old, 15×3–4 | 50 | 5.00 | 5×50 | self-preference; invalid |
| 2 | v1 / gpt-oss-20b (same family) | old | 23 | 5.00 | 5×23 | partial; invalid |
| 3 | v1 / qwen3.8-27b | old | 9 | 5.00 | 5×9 | smoke test, 3 distinct briefs |
| 4 | v1 / qwen3.8-27b | new, dev | 20 | 4.80 | 5×18, 4×1, 2×1 | **baseline** |
| 5 | v2 / qwen3.8-27b | new, dev | 20 | 4.95 | 5×19, 4×1 | accepted |
| 6 | v2 / qwen3.8-27b | new, **test** | 10 | **4.40** | 5×8, 2×2 | **final** |

### 5.3 System prompt v1 → v2

**v1** restates the style guide, requires JSON `{subject, email}`, and says to "sign as a plausible sender name". On the new dev briefs it covered every key point, reproduced every number and respected all 5 "do not…" constraints. By manual review, **7/20 emails added content not in the brief** (promises such as "we will notify you immediately", reasons, "as promised"). For the 15 briefs without a sender it invented 7 different names and several job titles. **v2** is identical to v1 except for three sections: NO ADDITIONS (use only the brief; a greeting, a short thank-you or apology, and one generic line are allowed), RECIPIENT NAME (exact name, never placeholders, role-based greeting if the name is unknown) and SENDER (sign exactly as given, otherwise a first name only).

| Metric (dev n=20; criteria set before the run) | v1 | v2 dev | v2 test (n=10) |
|---|---|---|---|
| Judge average score | 4.80 | 4.95 | 4.40 |
| Key points covered (judge) / tone rules passed (judge) | 20/20 / 20/20 | 20/20 / 20/20 | 10/10 / 10/10 |
| **Emails with content not in the brief (manual review)** | **7/20** | **0/20** | **0/10** |
| No-sender emails signed with a first name only | 4/15 | 15/15 | 8/9 |
| Unfilled placeholders (rule check) | 0/20 | 0/20 | 0/10 |
| Sub-5 scores that are judge false positives (correct_names) | 2/2 | 1/1 | 2/2 |

**How to read this:** the judge's 4.80 → 4.95 on dev is **not** evidence of improvement. The only two scores that changed belong to emails with the *same* greeting in both runs (judge noise). The locked judge already scored v1's added content as 5. v2's real gains are in the rows the judge cannot see, which we measured by review and by rule checks: **additions 7 → 0** and **sender compliance 4/15 → 15/15**. On test, the two 2s are judge false positives, and the one real defect is a "Manager" signature. With the judge's own errors corrected, the test split would be 10/10 at 5. So the fixes generalised beyond the dev examples quoted in the prompt. v2 introduced side effects that the judge also misses: a generic "let me know if you have any questions" line in 16/20 emails, names copied too literally ("Dear Prof. Mourad Ellouze"), and emails 30% shorter. The manual counts come from our own review and are pending confirmation by the two labellers.

## 6. Cost model

Groq list prices (27 Sep 2026): qwen3.8-27b $0.80 input / $4.00 output per 1M tokens; gpt-oss-120b $0.15 / $0.60. Token counts are measured per item from the runs. `scorer.py` still has zero prices, so the dollar figures below are computed by hand.

| Per item (v2, test run) | Tokens in / out | Latency avg / max | Cost per 1k items |
|---|---|---|---|
| System (writes the email) | 703 / 130 | 0.55 s / 1.6 s | $1.08 |
| Judge (grades the email) | 922 / 605 | 2.48 s / 7.9 s | $0.50 |
| **Total per 1,000 requests** | | | **$1.59** |
| **At 100× volume (100,000 requests)** | | | **≈ $159** |

Notes: v2's longer prompt adds about 267 input tokens per email, roughly +$0.21 per 1k. The judge costs about half as much as the system per item, even though its output is longer, because gpt-oss-120b is 5–7× cheaper per token. At 100× volume, the constraint is Groq's rate limits more than cost: two calls per request means spreading the load or moving to a higher tier. A judge validation run costs 35 calls on dev, and a full bias check costs 3 calls per item.